```
// Servidor MCP "Meta Business" da flamme.
// Publica e agenda posts no Facebook e no Instagram pelas APIs oficiais da Meta
//
// - Facebook: o agendamento é nativo (o post fica agendado dentro da própria Pág
// - Instagram: a API da Meta não aceita agendamento, então este servidor guarda
//   e publica sozinho na hora marcada (verificação a cada 30 segundos).

import express from "express";
import fs from "node:fs";
import path from "node:path";
import crypto from "node:crypto";
import { z } from "zod";
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StreamableHTTPServerTransport } from "@modelcontextprotocol/sdk/server/s

// ---------------------------------------------------------------------------
// Configuração
// ---------------------------------------------------------------------------
const PORT = Number(process.env.PORT || 3000);
const MCP_SECRET = process.env.MCP_SECRET;
const META_ACCESS_TOKEN = process.env.META_ACCESS_TOKEN;
const GRAPH_VERSION = process.env.GRAPH_VERSION || "v23.0";
const DATA_DIR = process.env.DATA_DIR || "./data";
const FUSO_PADRAO = process.env.FUSO_PADRAO || "-03:00"; // horário de Brasília

if (!MCP_SECRET || MCP_SECRET.length < 24) {
  console.error("Defina MCP_SECRET com pelo menos 24 caracteres (ex.: openssl ran
  process.exit(1);
}
if (!META_ACCESS_TOKEN) {
  console.error("Defina META_ACCESS_TOKEN (token do usuário do sistema no Meta Bu
  process.exit(1);
}

const GRAPH = `https://graph.facebook.com/${GRAPH_VERSION}`;

// ---------------------------------------------------------------------------
// Chamadas à Graph API
// ---------------------------------------------------------------------------
async function graph(caminho, { method = "GET", params = {}, token = META_ACCESS_
  const url = new URL(caminho.startsWith("http") ? caminho : `${GRAPH}/${caminho.
  const corpo = new URLSearchParams();
  const todos = { ...params, access_token: token };
  for (const [k, v] of Object.entries(todos)) {
```

```
    if (v === undefined || v === null) continue;
    const valor = typeof v === "object" ? JSON.stringify(v) : String(v);
    if (method === "GET" || method === "DELETE") url.searchParams.set(k, valor);
    else corpo.set(k, valor);
  }
  const resp = await fetch(url, {
    method,
    body: method === "GET" || method === "DELETE" ? undefined : corpo,
  });
  const dados = await resp.json().catch(() => ({}));
  if (!resp.ok || dados.error) {
    const e = dados.error || {};
    const msg = e.error_user_msg || e.message || `HTTP ${resp.status}`;
    throw new Error(`Meta API: ${msg}${e.code ? ` (código ${e.code}${e.error_subc
  }
  return dados;
}

const esperar = (ms) => new Promise((r) => setTimeout(r, ms));

// ---------------------------------------------------------------------------
// Contas (Páginas + Instagram conectados), com cache de 10 minutos
// ---------------------------------------------------------------------------
let cacheContas = { em: 0, contas: [] };

async function listarContas(forcar = false) {
  if (!forcar && Date.now() - cacheContas.em < 10 * 60 * 1000) return cacheContas
  const contas = [];
  let proxima = "me/accounts";
  let params = {
    fields: "id,name,access_token,instagram_business_account{id,username}",
    limit: 100,
  };
  while (proxima) {
    const r = await graph(proxima, { params });
    for (const p of r.data || []) {
      contas.push({
        page_id: p.id,
        pagina: p.name,
        page_token: p.access_token,
        ig_id: p.instagram_business_account?.id || null,
        instagram: p.instagram_business_account?.username || null,
      });
    }
    proxima = r.paging?.next || null;
    params = {}; // a URL "next" já traz os parâmetros
  }
```

```
  cacheContas = { em: Date.now(), contas };
  return contas;
}

const normalizar = (s) =>
  String(s || "")
    .toLowerCase()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .replace(/^@/, "")
    .trim();

// Aceita ID da Página, nome da Página ou @ do Instagram
async function acharConta(conta) {
  const contas = await listarContas();
  const alvo = normalizar(conta);
  const exata = contas.find(
    (c) => c.page_id === conta || c.ig_id === conta || normalizar(c.pagina) === a
  );
  if (exata) return exata;
  const parciais = contas.filter((c) => normalizar(c.pagina).includes(alvo) || no
  if (parciais.length === 1) return parciais[0];
  if (parciais.length > 1) {
    throw new Error(`"${conta}" corresponde a mais de uma conta: ${parciais.map((
  }
  throw new Error(`Conta "${conta}" não encontrada. Use listar_contas para ver as
}

// ---------------------------------------------------------------------------
// Datas
// ---------------------------------------------------------------------------
function lerDataHora(texto) {
  if (!texto || /^agora$/i.test(texto.trim())) return null;
  let s = texto.trim().replace(" ", "T");
  if (!/([zZ]|[+-]\d{2}:?\d{2})$/.test(s)) {
    if (/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}$/.test(s)) s += ":00";
    s += FUSO_PADRAO;
  }
  const d = new Date(s);
  if (Number.isNaN(d.getTime())) throw new Error(`Data/hora inválida: "${texto}".
  return d;
}

const formatarBR = (d) =>
  new Date(d).toLocaleString("pt-BR", { timeZone: "America/Sao_Paulo", dateStyle:

// ---------------------------------------------------------------------------
```

```
// Verificação das URLs de mídia (a Meta precisa baixar o arquivo por um link púb
// ---------------------------------------------------------------------------
async function verificarMidia(urls) {
  const avisos = [];
  for (const url of urls) {
    try {
      let r = await fetch(url, { method: "HEAD", redirect: "follow" });
      if (!r.ok || !r.headers.get("content-type")) {
        r = await fetch(url, { method: "GET", headers: { Range: "bytes=0-0" }, re
      }
      const tipo = r.headers.get("content-type") || "";
      if (!r.ok) avisos.push(`${url} respondeu HTTP ${r.status}: o link precisa s
      else if (!/^(image|video)\//.test(tipo))
        avisos.push(`${url} não parece ser um arquivo direto de imagem/vídeo (tip
    } catch (e) {
      avisos.push(`Não consegui acessar ${url}: ${e.message}`);
    }
  }
  return avisos;
}

// ---------------------------------------------------------------------------
// Armazenamento dos agendamentos do Instagram (arquivo JSON no volume)
// ---------------------------------------------------------------------------
fs.mkdirSync(DATA_DIR, { recursive: true });
const ARQUIVO = path.join(DATA_DIR, "agendamentos-instagram.json");

function carregar() {
  try {
    return JSON.parse(fs.readFileSync(ARQUIVO, "utf8"));
  } catch {
    return [];
  }
}
function salvar(lista) {
  const tmp = `${ARQUIVO}.tmp`;
  fs.writeFileSync(tmp, JSON.stringify(lista, null, 2));
  fs.renameSync(tmp, ARQUIVO);
}
function atualizar(id, mudancas) {
  const lista = carregar();
  const item = lista.find((x) => x.id === id);
  if (item) Object.assign(item, mudancas);
  salvar(lista);
  return item;
}
```

```
// Se o servidor caiu no meio de uma publicação, não tenta de novo (evita post du
{
  const lista = carregar();
  let mudou = false;
  for (const item of lista) {
    if (item.status === "publicando") {
      item.status = "erro";
      item.erro = "Publicação interrompida por reinício do servidor. Confira no I
      mudou = true;
    }
  }
  if (mudou) salvar(lista);
}

// ---------------------------------------------------------------------------
// Publicação no Instagram
// ---------------------------------------------------------------------------
const ehVideo = (url) => /\.(mp4|mov|m4v)(\?|$)/i.test(url);

async function aguardarContainer(id, token, limiteMs = 10 * 60 * 1000) {
  const inicio = Date.now();
  while (Date.now() - inicio < limiteMs) {
    const r = await graph(id, { params: { fields: "status_code,status" }, token }
    if (r.status_code === "FINISHED") return;
    if (r.status_code === "ERROR" || r.status_code === "EXPIRED")
      throw new Error(`A Meta não conseguiu processar a mídia (${r.status || r.st
    await esperar(5000);
  }
  throw new Error("Tempo esgotado esperando a Meta processar a mídia.");
}

async function publicarInstagram({ ig_id, page_token, tipo, legenda, midias, capa
  const criar = (params) => graph(`${ig_id}/media`, { method: "POST", params, tok
  let container;

  if (tipo === "imagem") {
    container = await criar({ image_url: midias[0], caption: legenda });
  } else if (tipo === "reels") {
    container = await criar({ media_type: "REELS", video_url: midias[0], caption:
  } else if (tipo === "story") {
    container = await criar(ehVideo(midias[0]) ? { media_type: "STORIES", video_u
  } else if (tipo === "carrossel") {
    const filhos = [];
    for (const url of midias) {
      const f = await criar(ehVideo(url) ? { media_type: "VIDEO", video_url: url,
      await aguardarContainer(f.id, page_token);
      filhos.push(f.id);
```

```
    }
    container = await criar({ media_type: "CAROUSEL", children: filhos.join(","),
  } else {
    throw new Error(`Tipo de post desconhecido: ${tipo}`);
  }

  await aguardarContainer(container.id, page_token);
  const publicado = await graph(`${ig_id}/media_publish`, { method: "POST", param
  let link = null;
  try {
    link = (await graph(publicado.id, { params: { fields: "permalink" }, token: p
  } catch {}
  return { media_id: publicado.id, link };
}

// Verifica a cada 30 segundos se há posts do Instagram para publicar
let rodando = false;
setInterval(async () => {
  if (rodando) return;
  rodando = true;
  try {
    const devidos = carregar().filter((x) => x.status === "pendente" && new Date(
    for (const item of devidos) {
      atualizar(item.id, { status: "publicando" });
      try {
        const conta = await acharConta(item.ig_id);
        const r = await publicarInstagram({ ...item, page_token: conta.page_token
        atualizar(item.id, { status: "publicado", publicado_em: new Date().toISOS
        console.log(`[instagram] publicado ${item.id} em @${item.instagram}`);
      } catch (e) {
        atualizar(item.id, { status: "erro", erro: e.message });
        console.error(`[instagram] erro em ${item.id}: ${e.message}`);
      }
    }
  } finally {
    rodando = false;
  }
}, 30 * 1000);

// ---------------------------------------------------------------------------
// Publicação no Facebook (agendamento nativo da Página)
// ---------------------------------------------------------------------------
async function publicarFacebook({ page_id, page_token, tipo, mensagem, link, midi
  const agendamento = quando
    ? { published: false, scheduled_publish_time: Math.floor(quando.getTime() / 1
    : { published: true };
  const post = (caminho, params) => graph(`${page_id}/${caminho}`, { method: "POS
```

```
  if (tipo === "texto") return post("feed", { message: mensagem, link, ...agendam
  if (tipo === "foto") return post("photos", { url: midias[0], caption: mensagem,
  if (tipo === "video") return post("videos", { file_url: midias[0], description:
  if (tipo === "fotos") {
    const ids = [];
    for (const url of midias) {
      const f = await post("photos", { url, published: false, temporary: Boolean(
      ids.push({ media_fbid: f.id });
    }
    return post("feed", { message: mensagem, attached_media: ids, ...agendamento
  }
  throw new Error(`Tipo de post desconhecido: ${tipo}`);
}

// ---------------------------------------------------------------------------
// Ferramentas MCP
// ---------------------------------------------------------------------------
const ok = (dados) => ({ content: [{ type: "text", text: typeof dados === "string
const falha = (e) => ({ isError: true, content: [{ type: "text", text: e.message
const seguro = (fn) => async (args) => {
  try {
    return await fn(args);
  } catch (e) {
    return falha(e);
  }
};

const DESC_DATA =
  "Data e hora no horário de Brasília, formato 2026-11-03T18:00. Deixe vazio ou u
const DESC_CONTA = "ID da Página, nome da Página ou @ do Instagram do cliente (ex
const DESC_MIDIAS =
  "Links públicos e diretos dos arquivos (a Meta baixa o arquivo pelo link). Link

function criarServidor() {
  const server = new McpServer({ name: "meta-business-flamme", version: "1.0.0" }

  server.registerTool(
    "listar_contas",
    {
      title: "Listar contas",
      description: "Lista as Páginas do Facebook e os Instagrams conectados que o
      inputSchema: { atualizar: z.boolean().optional().describe("true para ignora
    },
    seguro(async ({ atualizar }) => {
      const contas = await listarContas(Boolean(atualizar));
      return ok(contas.map(({ page_token, ...c }) => c));
```

```
  })
);

server.registerTool(
  "agendar_post_instagram",
  {
    title: "Agendar post no Instagram",
    description:
      "Publica ou agenda um post no Instagram do cliente: imagem, carrossel (2
    inputSchema: {
      conta: z.string().describe(DESC_CONTA),
      tipo: z.enum(["imagem", "carrossel", "reels", "story"]),
      legenda: z.string().max(2200).optional().describe("Legenda com hashtags (
      midias: z.array(z.string().url()).min(1).max(10).describe(DESC_MIDIAS),
      capa_url: z.string().url().optional().describe("Capa do reels (imagem, li
      data_hora: z.string().optional().describe(DESC_DATA),
    },
  },
  seguro(async ({ conta, tipo, legenda, midias, capa_url, data_hora }) => {
    const c = await acharConta(conta);
    if (!c.ig_id) throw new Error(`A Página "${c.pagina}" não tem Instagram pro
    if (tipo === "carrossel" && midias.length < 2) throw new Error("Carrossel p
    if (tipo !== "carrossel" && midias.length > 1) throw new Error(`O tipo "${t
    const quando = lerDataHora(data_hora);
    if (quando && quando < new Date()) throw new Error(`O horário ${formatarBR(
    const avisos = await verificarMidia(capa_url ? [...midias, capa_url] : midi

    if (!quando) {
      const r = await publicarInstagram({ ig_id: c.ig_id, page_token: c.page_to
      return ok({ status: "publicado", instagram: `@${c.instagram}`, ...r, avis
    }

    const item = {
      id: `ig_${crypto.randomBytes(5).toString("hex")}`,
      status: "pendente",
      ig_id: c.ig_id,
      instagram: c.instagram,
      pagina: c.pagina,
      tipo,
      legenda,
      midias,
      capa_url,
      quando: quando.toISOString(),
      criado_em: new Date().toISOString(),
    };
    salvar([...carregar(), item]);
    return ok({ status: "agendado", id: item.id, instagram: `@${c.instagram}`,
```

```
  })
);

server.registerTool(
  "agendar_post_facebook",
  {
    title: "Agendar post no Facebook",
    description:
      "Publica ou agenda um post na Página do Facebook do cliente: texto (com l
    inputSchema: {
      conta: z.string().describe(DESC_CONTA),
      tipo: z.enum(["texto", "foto", "fotos", "video"]),
      mensagem: z.string().optional().describe("Texto do post."),
      link: z.string().url().optional().describe("Link para compartilhar (só no
      titulo: z.string().optional().describe("Título do vídeo (só no tipo video
      midias: z.array(z.string().url()).max(10).optional().describe(DESC_MIDIAS
      data_hora: z.string().optional().describe(DESC_DATA),
    },
  },
  seguro(async ({ conta, tipo, mensagem, link, titulo, midias = [], data_hora }
    const c = await acharConta(conta);
    if (tipo !== "texto" && midias.length === 0) throw new Error(`O tipo "${tip
    if (tipo === "texto" && !mensagem && !link) throw new Error("Post de texto
    const quando = lerDataHora(data_hora);
    if (quando && quando.getTime() - Date.now() < 10 * 60 * 1000)
      throw new Error("No Facebook, o agendamento precisa ser pelo menos 10 min
    const avisos = midias.length ? await verificarMidia(midias) : [];
    const r = await publicarFacebook({ page_id: c.page_id, page_token: c.page_t
    return ok({
      status: quando ? "agendado" : "publicado",
      id: r.post_id || r.id,
      pagina: c.pagina,
      quando: quando ? formatarBR(quando) : "agora",
      avisos,
    });
  })
);

server.registerTool(
  "listar_agendados",
  {
    title: "Listar agendados",
    description:
      "Lista os posts agendados: os do Facebook (direto da Página) e os do Inst
    inputSchema: {
      conta: z.string().optional().describe("Filtrar por cliente. " + DESC_CONT
      incluir_publicados: z.boolean().optional().describe("true para incluir os
```

```
    },
  },
  seguro(async ({ conta, incluir_publicados }) => {
    const contas = conta ? [await acharConta(conta)] : await listarContas();
    const resultado = { facebook: [], instagram: [] };

    for (const c of contas) {
      try {
        const r = await graph(`${c.page_id}/scheduled_posts`, {
          params: { fields: "id,message,scheduled_publish_time,permalink_url",
          token: c.page_token,
        });
        for (const p of r.data || []) {
          resultado.facebook.push({
            id: p.id,
            pagina: c.pagina,
            quando: p.scheduled_publish_time ? formatarBR(new Date(p.scheduled_
            mensagem: p.message?.slice(0, 120) || "",
          });
        }
      } catch (e) {
        resultado.facebook.push({ pagina: c.pagina, erro: e.message });
      }
    }

    const igIds = new Set(contas.map((c) => c.ig_id).filter(Boolean));
    resultado.instagram = carregar()
      .filter((x) => igIds.has(x.ig_id))
      .filter((x) => incluir_publicados || x.status === "pendente")
      .sort((a, b) => new Date(a.quando) - new Date(b.quando))
      .map((x) => ({
        id: x.id,
        status: x.status,
        instagram: `@${x.instagram}`,
        tipo: x.tipo,
        quando: formatarBR(x.quando),
        legenda: x.legenda?.slice(0, 120) || "",
        link: x.link,
        erro: x.erro,
      }));
    return ok(resultado);
  })
);

server.registerTool(
  "cancelar_agendamento",
  {
```

```
      title: "Cancelar agendamento",
      description:
        "Cancela um post agendado. Para Instagram, use o id que começa com 'ig_'.
      inputSchema: {
        id: z.string(),
        conta: z.string().optional().describe("Obrigatório para posts do Facebook
      },
    },
    seguro(async ({ id, conta }) => {
      if (id.startsWith("ig_")) {
        const lista = carregar();
        const item = lista.find((x) => x.id === id);
        if (!item) throw new Error(`Agendamento ${id} não encontrado.`);
        if (item.status !== "pendente") throw new Error(`Esse post está com statu
        salvar(lista.filter((x) => x.id !== id));
        return ok(`Agendamento ${id} (@${item.instagram}, ${formatarBR(item.quand
      }
      if (!conta) throw new Error("Para cancelar um post do Facebook, informe a c
      const c = await acharConta(conta);
      await graph(id, { method: "DELETE", token: c.page_token });
      return ok(`Post ${id} da Página "${c.pagina}" cancelado.`);
    })
  );

  return server;
}

// ---------------------------------------------------------------------------
// HTTP
// ---------------------------------------------------------------------------
const app = express();
app.use(express.json({ limit: "2mb" }));

app.get("/", (_req, res) => res.send("Meta Business MCP da flamme. está no ar."))

const ROTA = `/mcp/${MCP_SECRET}`;

app.post(ROTA, async (req, res) => {
  const server = criarServidor();
  const transport = new StreamableHTTPServerTransport({ sessionIdGenerator: undef
  res.on("close", () => {
    transport.close();
    server.close();
  });
  try {
    await server.connect(transport);
    await transport.handleRequest(req, res, req.body);
```

```
  } catch (e) {
    console.error(e);
    if (!res.headersSent) res.status(500).json({ jsonrpc: "2.0", error: { code: -
  }
});

const naoPermitido = (_req, res) =>
  res.status(405).json({ jsonrpc: "2.0", error: { code: -32000, message: "Método
app.get(ROTA, naoPermitido);
app.delete(ROTA, naoPermitido);

app.listen(PORT, () => console.log(`Meta Business MCP rodando na porta ${PORT}`))
```